

Full-Stack Engineering Technical Exercise

KAD Global Agency | Confidential

Role:	Full-Stack Engineer
Technology:	Backend: Python 3.10+ (required). HTTP framework of your choice — FastAPI is recommended for clean async Socket.IO integration; Flask + flask-socketio is also accepted. Frontend: Next.js (App Router or Pages Router) with React and TypeScript (required). Database: your choice (SQLite, PostgreSQL, in-memory).
Estimated Time:	5–7 hours
Submission:	Source code (Git repository or zip) + README + <code>docker compose up</code> as the run command + sample <code>npx playwright test</code> output
Send to:	muhammed.khan@truxel.ai, monica@kad.agency

1. Objective

Build a small, correct full-stack application: an HTTP API + a Next.js single-page UI that lets an authenticated operator submit and monitor jobs targeting physical assets. The backend drives a state machine that progresses each job through `PENDING → RUNNING → COMPLETED` (or `FAILED`) on its own internal timer, and pushes live state updates to the operator's UI over Socket.IO.

We are evaluating **API design**, **state-machine modelling**, **authentication & authorisation** across HTTP and WebSocket, **real-time UI** (Socket.IO client wired to React), **form validation and error UX**, **componentisation**, **TypeScript discipline**, and **containerised packaging**. We are not evaluating elaborate visual design or production-grade infrastructure.

2. Scenario

An operator logs in to a small web UI by pasting an API token. They see a list of their own jobs, each showing its current state. They can submit a new job for a future time window targeting one of the available assets. The backend accepts the job (state `PENDING`), then on its own internal timer transitions it to `RUNNING`, then to `COMPLETED` (or `FAILED` for the designated fault asset). Each transition is pushed to that operator's UI in real time over Socket.IO; the operator sees the state change without reloading.

The system has three layers:

- **Backend** (your work) — HTTP API + Socket.IO server. Owns persistence and the state machine. Authenticates every request and every Socket.IO connection.
- **Frontend** (your work) — Next.js single-page UI. Authenticates by token, renders the operator's jobs, drives a create-job form, and re-renders on Socket.IO events.

- **Operator** — a human (and, for grading, a Playwright harness) who interacts only with the UI.

3. Provided Components

The exercise ships with everything you need to develop and self-assess against:

```
fs-exercise-bundle/
├── fullstack-engineer-exercise.pdf  # this document
├── README.md                       # candidate-facing guide
├── data/
│   ├── seed.json                  # users (with API tokens) + assets
├── harness/
│   ├── package.json               # Playwright + deps
│   ├── playwright.config.ts
│   └── verify.spec.ts             # self-assessment harness
```

3.1 Seed data (`data/seed.json`)

Three users, each with a static Bearer token:

user_id	name	token
user-001	Alice Operator	tok_alice_a1b2c3d4e5f6
user-002	Bob Operator	tok_bob_b2c3d4e5f6a1
user-003	Carol Operator	tok_carol_c3d4e5f6a1b2

Eight assets plus `asset-fault`, a designated failure asset that the backend must transition to `FAILED` instead of `COMPLETED` (see §4.4). The full asset list and operation enum live in `seed.json`.

3.2 Self-assessment harness (`harness/verify.spec.ts`)

A ready-to-run Playwright suite. After your application is up via `docker compose up -d`, run `cd harness && npm install && npx playwright test` and it reports PASS / FAIL across 19 scenarios covering every functional requirement below. If everything passes, you can be confident your implementation meets the specification.

The harness drives your UI through a browser. For it to find the right elements, your UI must include the `data-testid` attributes listed in §5.4. The harness will not pass without them.

4. Backend Requirements

4.1 HTTP REST API

Expose these endpoints on port `8000` . All endpoints under `/api/` require Bearer-token authentication (§4.3). `/health` does not require auth.

Method & path	Description
GET <code>/health</code>	Liveness check. Returns 200 with <code>{"status":"ok"}</code> .
POST <code>/api/jobs</code>	Submit a new job. Body per §4.2.
GET <code>/api/jobs/{id}</code>	Read a single job. Returns 404 if the job does not exist or belongs to another user. Response is a single job object (§4.2).
GET <code>/api/jobs</code>	List the caller's jobs (only). Response is a top-level JSON array of job objects, ordered newest-first by <code>created_at</code> . 200 always; empty array <code>[]</code> if none.

HTTP status code matrix:

Status	When
201 Created	POST <code>/api/jobs</code> succeeded.
200 OK	GET endpoints returning data; health check.
400 Bad Request	Malformed request (invalid JSON, missing required field, malformed ISO-8601 timestamp).
401 Unauthorized	Missing or invalid <code>Authorization</code> header.
404 Not Found	GET on a job that does not exist or belongs to another user. (Returning 403 is also acceptable.)
422 Unprocessable Entity	Body parses but violates a business rule (§4.2).

4.2 Request/response shapes & business rules

Request body for POST `/api/jobs` :

```
{
  "asset_id": "asset-001",
  "operation": "charge",
  "start_time": "2026-05-19T14:00:00.000Z",
  "end_time": "2026-05-19T15:00:00.000Z"
}
```

Response body for job objects (used by both job endpoints):

```
{
  "id": "uuid",
  "asset_id": "asset-001",
  "operation": "charge",
  "start_time": "2026-05-19T14:00:00.000Z",
  "end_time": "2026-05-19T15:00:00.000Z",
  "state": "PENDING", // see §4.4
  "error_message": null,
  "created_at": "2026-05-18T10:00:00.000Z",
  "updated_at": "2026-05-18T10:00:00.000Z"
}
```

Malformed input → 400:

- `start_time` or `end_time` is not parseable as ISO-8601 UTC with `Z` or `+00:00` suffix.
- Body is not valid JSON, or a required field is missing.

Business rules → 422:

- `asset_id` must exist in `seed.json`.
- `operation` is a non-empty string up to 64 chars. No enum enforcement required; `seed.json` lists suggested values.
- `start_time < end_time`.
- `start_time` must be strictly greater than the server's `now` at the moment of request handling (no minimum buffer required).
- Duration `end_time - start_time` must be between 15 minutes and 4 hours inclusive.

Error response shape: all 400, 401, 404, and 422 responses from `/api/*` must return a JSON body of the form:

```
{ "error": "human-readable string" }
```

The frontend renders that `error` field directly into `create-job-error`. *Note for FastAPI users: this means overriding the default Pydantic validation error format with a custom exception handler.*

What's *not* required: no idempotency keys, no atomic overlap detection between jobs on the same asset, no per-user active-job cap. Two operators may submit overlapping windows on the same asset and both will succeed — that's fine for this exercise.

4.3 Authentication & authorisation

- Every `/api/*` endpoint requires header `Authorization: Bearer <token>`.
- Valid tokens are the ones listed in `data/seed.json`. Tokens are static fixtures — treat them like test secrets, not production credentials.
- Each request maps to a single `user_id`. Read endpoints must scope results to that user only. A user attempting to read another user's job must receive 404 (or 403 if you prefer; not 200).
- The Socket.IO server (§4.5) must authenticate the same way and must scope all emits per user.

4.4 State machine

A job is always in exactly one of these four states:

State	Meaning	Entered by
PENDING	Accepted by the API, waiting for the backend to start processing.	Initial state on successful <code>POST /api/jobs</code> .
RUNNING	Backend is processing.	Internal timer transition from PENDING.
COMPLETED	Backend finished successfully.	Internal timer transition from RUNNING when <code>asset_id != "asset-fault"</code> .
FAILED	Backend reported a failure. <code>error_message</code> is populated.	Internal timer transition from RUNNING when <code>asset_id == "asset-fault"</code> . Use <code>error_message "Simulated failure (asset-fault)"</code> .

Allowed transitions (the only ones the backend should produce):

From	To	After (default)
PENDING	RUNNING	1 second
RUNNING	COMPLETED or FAILED	1 second after entering RUNNING
COMPLETED	— (terminal)	—
FAILED	— (terminal)	—

Both delays must be configurable via environment variables `TRANSITION_TO_RUNNING_SECONDS` and `TRANSITION_TO_TERMINAL_SECONDS` (defaults `1.0` each). The harness expects total time from submission to terminal state to be under 5 seconds, so leave the defaults short.

Transitions are driven only by the backend's internal timer. The HTTP API does not expose a way to mutate state directly. You may implement the timer with `asyncio.create_task + asyncio.sleep`, a scheduler such as `APScheduler`, or `threads` — your choice. If you use `threads` or an external scheduler, remember that `sio.emit` must be scheduled onto the `asyncio` event loop (e.g. via `asyncio.run_coroutine_threadsafe`).

Persistence across restarts of in-flight (PENDING / RUNNING) jobs is **not** required for the harness to pass. If your service restarts, jobs that were active at the time may be left in their last persisted state forever — acceptable. Mention this caveat in the README if it applies.

4.5 Socket.IO real-time updates

Use the **Socket.IO protocol** (the same one used by `python-socketio` and `flask-socketio`). **Mount the Socket.IO server at the default path `/socket.io/`** — the harness connects there.

- **Auth:** the client connects with the token as a query-string parameter, e.g. `/socket.io/?token=<bearer>`. Reject the connection if the token is missing or invalid.
- **Scoping:** on successful connection, put the socket into a per-user "room" (Socket.IO room name is your choice; `user:<user_id>` is a sensible default).

- **Event:** when a job changes state, emit a `job_update` event *only* to that job's owner's room.
Payload:

```
{
  "job_id": "uuid",
  "state": "RUNNING",
  "error_message": null,
  "timestamp": "2026-05-18T10:00:01.000Z"
}
```

- A user must **not** receive events for jobs they don't own. The harness tests this directly.

4.6 CORS

Your backend is reached cross-origin by the frontend (the browser loads the page from `localhost:3000` and calls the API at `localhost:8000`). You must configure CORS on the backend such that:

- The frontend's origin can `POST` and `GET` to `/api/*`.
- `OPTIONS` preflights succeed.
- The `Authorization` and `Content-Type` headers are permitted.

In FastAPI, the one-liner is `CORSMiddleware`; in Flask, `flask-cors`. **Note:** `python-socketio`'s server has its own CORS option (`cors_allowed_origins`) — the HTTP CORS middleware does not cover the Socket.IO handshake. Configure both. Without them, the browser will block your API calls and your Socket.IO upgrade.

4.7 What you are *not* asked to do on the backend

- No MQTT, no external worker.
- No cancel endpoint.
- No idempotency keys.
- No atomic overlap detection.
- No rate limiting, no metrics endpoint.
- No production-grade auth: static Bearer tokens are sufficient.

5. Frontend Requirements

5.1 Stack

- **Next.js** (App Router or Pages Router, your choice).
- **React 18+**, **TypeScript**. Plain JavaScript is not accepted.
- Styling: your choice (CSS modules, Tailwind, plain CSS, MUI, etc.). We are not grading visual polish.
- Socket.IO client: `socket.io-client` v4.x.
- Frontend must be served on port `3000`.

5.2 Pages

Route	Purpose
<code>/login</code>	Token entry page. On submit with a non-empty token, store it (localStorage or cookie), then route to <code>/</code> . On submit with an empty token, render <code>login-error</code> and do not navigate. You do <i>not</i> need to validate the token against the backend at this point — an invalid token will be rejected by the first authenticated request on the dashboard, which then redirects back to <code>/login</code> .
<code>/</code>	Authenticated dashboard. If no token is present, redirect to <code>/login</code> . Shows the create-job form (§5.3) and the live job list (§5.4). The <code>logout-button</code> must be visible immediately when the page renders.

5.3 Create-job form

The form is rendered on the dashboard (either inline or behind a "New job" button — your choice; the harness will look for it on `/`).

- Fields: `asset_id`, `operation`, `start_time`, `end_time`.
- **All four inputs must be `<input type="text">`** (not `<select>`, not datetime pickers). This keeps the harness deterministic; Playwright's `.fill()` does not work on `<select>`.
- `start_time` and `end_time` must accept ISO-8601 UTC strings with the canonical `.000Z` suffix (e.g. `2026-05-19T14:00:00.000Z`) — this is exactly the format the harness sends.
- On submit, POST to your backend. On 201, append the new job (from the response body, which is a full job object per §4.2) to the *top* of the list and reset the form.
- On a 422 or 400 response, parse the JSON body's `error` field (§4.2) and render it in the element with `data-testid="create-job-error"`. Do not navigate away.
- On a 401 response, clear the stored token and redirect to `/login`.

5.4 Live job list & required `data-testid` attributes

The list must render every job returned by `GET /api/jobs` on page load, ordered newest first. It must subscribe to the Socket.IO server and update each row's state when a matching `job_update` event arrives. A row added by the create-job form must subsequently update via Socket.IO without a manual refresh.

Socket.IO client connection: the frontend must connect to the backend's Socket.IO server at path `/socket.io/` with the user's token as the `token` query-string parameter — the same handshake described in §4.5. Subscribe to `job_update` events and merge them into the list's state, matching by `job_id`.

Required `data-testid` attributes (the harness depends on these **exactly** as shown):

testid	On	Notes
login-token-input	token input on <code>/login</code>	plain text input
login-submit	submit button on <code>/login</code>	button
login-error	error container on <code>/login</code>	visible when login fails (e.g. empty token or 401 on a follow-up call)
logout-button	logout button on <code>/</code>	clears token, routes to <code>/login</code>
create-job-asset	<code>asset_id</code> text input on <code>/</code>	must be <code><input type="text"></code>
create-job-operation	<code>operation</code> text input on <code>/</code>	must be <code><input type="text"></code>
create-job-start	<code>start_time</code> text input on <code>/</code>	plain text, ISO-8601 UTC, <code>.000Z</code> suffix
create-job-end	<code>end_time</code> text input on <code>/</code>	plain text, ISO-8601 UTC, <code>.000Z</code> suffix
create-job-submit	submit button on <code>/</code>	—
create-job-error	error container near the form	holds server-side validation messages; empty/hidden when no error
job-list	container element wrapping all job rows	—
job-row	each row in the list	must additionally carry <code>data-job-id="<job uuid>"</code> on the same element
job-state	state text/badge inside each row	must contain the literal state string (PENDING, RUNNING, COMPLETED, FAILED) as its text content
job-error	<code>error_message</code> text inside each row	present and non-empty when state is FAILED; may be absent or empty for non-FAILED rows

5.5 What you are *not* asked to do on the frontend

- No SSR/SSG of authenticated content — client-side rendering is fine.
- No optimistic UI (don't render the row before the 201; the harness asserts the row appears *after* the response).
- No pagination, sorting controls, search, or filters.
- No edit/cancel/delete flows for jobs.
- No visual design grading. A spreadsheet-looking table is fine.

- No accessibility audit (use sane semantic HTML and we're happy).

6. Run & Packaging Requirements

6.1 `docker compose up` must boot the entire app

Your submission must include a top-level `docker-compose.yml` such that `docker compose up -d` (in the project root) boots the backend on port `8000` and the frontend on port `3000`, and the application is ready to receive traffic within ~60 seconds. The Playwright harness will be run against this composition.

This implies:

- A `Dockerfile` for the backend.
- A `Dockerfile` for the frontend (production build is fine; `next dev` is also acceptable).
- The frontend must reach the backend at a URL configurable via environment variable. The default in the compose file should work without any host edits.
- No external services (no MQTT broker, no separate worker, no third-party API). The backend is self-contained.

6.2 Makefile

```
make setup      # build images
make run        # docker compose up -d
make stop       # docker compose down
make test       # cd harness && npm install && npx playwright test
```

6.3 Configuration

- Configuration via environment variables only; no hard-coded paths or secrets.
- The backend reads `data/seed.json` at startup; bind-mount it from the bundle if you wish, or copy it into the image.
- Sensible defaults: backend on `:8000`, frontend on `:3000`, transition timers at 1 second each.

7. Technical Expectations

- **TypeScript** for the frontend, with `strict` mode (or close to it) enabled in `tsconfig.json`. No `any` sprinkled to silence the compiler.
- **Modular layout**: separate concerns. Backend: routing / validation / persistence / state-machine. Frontend: pages / components / hooks / api-client. Single-file solutions are accepted but harder to review.
- **Tests — at least three of your own**, in addition to the shipped Playwright harness. Examples (any three of these or your own):
 - Backend unit test: a job submitted in the past returns 422.
 - Backend integration test: a submitted job reaches `COMPLETED` within ~3 seconds.
 - Backend authz test: user A cannot read user B's job.
 - Frontend component test: the job-row component renders the `FAILED` `error_message`.
 - Frontend hook test: the Socket.IO hook reconnects on disconnect.

- **Structured logging** in the backend (one JSON object per line, or equivalent) at request boundaries.
- Pin dependencies (`requirements.txt` / `pyproject.toml` on the backend; `package-lock.json` on the frontend).
- The full `harness/verify.spec.ts` Playwright suite must pass against your running application.

8. Documentation (Required)

Include a `README.md` at the project root covering, concisely:

- **How to run** — the `docker compose up` sequence; how to run your tests; how to run the harness.
- **Architecture** — one or two paragraphs describing backend layers and frontend module layout. A small diagram is welcome but not required.
- **State-machine implementation** — how you schedule the timer-driven transitions and what happens on restart.
- **Authentication strategy** — where the token is stored on the client, how it's sent on HTTP requests and on the Socket.IO handshake.
- **Trade-offs** — what you cut for the 5–7h budget, what you would build next.

Aim for two to four pages. We value concision and honest reasoning over completeness.

9. Submission

- Source code as a Git repository (preferred) or zip.
- `README.md` per §8.
- `Makefile` per §6.2.
- `docker-compose.yml` at the project root.
- Paste the output of a successful `npx playwright test` run into the README or a separate `verify-output.txt`.

10. Evaluation Criteria

Criteria	What we look for
End-to-end correctness	All scenarios in <code>harness/verify.spec.ts</code> pass against your running app.
State-machine discipline	Transitions fire on the timer, terminal states do not transition further, <code>error_message</code> is populated only on FAILED.
Authentication & authorisation	Tokens enforced on every <code>/api</code> path and on Socket.IO; cross-user reads return 404 / 403; Socket.IO events are scoped per user.
Real-time UI	Job rows update on Socket.IO events without manual refresh; transitions render in order.
Form & validation UX	Validation errors from the backend are surfaced in the UI; the form does not lose user input on an error.

Criteria	What we look for
TypeScript discipline	Strict-mode-friendly types; minimal <code>any</code> ; shared types between client and server are explicit.
Code structure	Clear separation of concerns on both sides; small testable units; readable.
Containerised packaging	<code>docker compose up</code> boots a working app from a clean clone with no manual edits.
Testing	The three required tests exist, pass, and exercise the spec rather than smoke-tests.
Documentation	README answers the required questions with honest trade-off discussion.

Good luck. We look forward to reviewing your submission.

— KAD Global Agency, Platform Engineering Team